

Reti di Calcolatori

Appello del 24/01/2025

Compito 1

Cognome:
Nome:
Matricola:

Tempo a disposizione: 3h

È obbligatorio inserire Cognome Nome Matricola e Numero Compito all'inizio di ogni file sorgente, pena la non valutazione del compito, che viene stampato in modo automatico solo in caso siano presenti gli elementi detti sopra.

Si devono consegnare **tutti i file sorgente e tutti gli eseguibili prodotti singolarmente** (per favore, solo quelli relativi ai file sorgente consegnati!!!).

La prova intende valutare le capacità progettuali e di programmazione sia in **ambiente Java** che in **ambiente C**, pertanto è consigliato sviluppare **entrambe** le soluzioni richieste al meglio. *In entrambi gli esercizi, sia in Java che in C, si effettuino gli opportuni controlli sui parametri della richiesta e si gestiscano le eccezioni, tenendo presente i criteri secondo cui si possa ripristinare il funzionamento del programma oppure si debba forzarne la terminazione. Leggete con attenzione le specifiche del problema prima di impegnarvi "a testa bassa" nello sviluppo delle singole parti. Naturalmente, ci aspettiamo che i componenti da consegnare siano stati provati e siano funzionanti.*

Si richiede la progettazione e la realizzazione di servizi invocabili da parte di più clienti sul file system di una macchina server (direttorio remoto). I file binari possono avere qualsiasi estensione (non .txt) e sono composti da dati binari. I file di testo sono i file con nome con suffisso "txt" e composti solo da linee di un numero definito di caratteri ASCII.

In particolare si vogliono realizzare i seguenti servizi:

1. **Conteggio delle linee in file testo di direttorio remoto.** Si devono contare le linee del file testo del direttorio remoto che iniziano con una lettera maiuscola e contengono esattamente uno specificato numero di occorrenze del carattere indicato dall'utente: questa operazione richiede all'utente un carattere e il numero di occorrenze, poi scandisce ciascun file di testo riga per riga contando il numero di righe che iniziano con una lettera maiuscola e contengono (in qualsiasi posizione della riga) un numero di occorrenze del carattere uguale a quelle indicate dall'utente; infine, restituisce a console il contatore esito dell'operazione.
2. **Eliminazione di tutte le occorrenze di caratteri numerici all'interno di un file di testo:** questa operazione richiede all'utente il nome di un file, elimina tutte le occorrenze di caratteri numerici (da '0' a '9') dal file stesso, e visualizza sul cliente a video il numero di eliminazioni effettuate dal server o un'indicazione di errore.
3. **Lista dei file di testo di un direttorio specificato dall'utente:** questa operazione richiede all'utente un nome di direttorio, quindi visualizza la lista dei nomi dei file di testo (con suffisso ".txt") contenuti nel direttorio richiesto.
4. **Trasferimento dal server al client di tutti i file binari di un direttorio e di tutti i suoi sottodirettori di primo livello:** questa operazione richiede all'utente il nome del direttorio e trasferisce tutti i file binari (i cui nomi non abbiano suffisso ".txt") contenuti all'interno nel direttorio indicato e in tutti i suoi sottodirettori di primo livello, salvandone il contenuto sul direttorio corrente del client.

Si richiede inoltre di **non** usare comandi Unix (es. il comando **ls**), ma solo primitive di sistema (es. **opendir()** in C) o funzioni di libreria (es. metodi della classe **File** in Java). Inoltre, si considerino e si segnalino le possibilità di **interferenze** fra le operazioni, evitandole dove necessario

Parte Java

Utilizzando **java RMI** sviluppare un'applicazione C/S che consenta di effettuare le operazioni remote per:

- **eliminare tutte le occorrenze di caratteri numerici all'interno di un file di testo;**
ricevere la lista dei nomi dei file di testo di un direttorio specificato dall'utente.

Il progetto RMI si basa su un'interfaccia (contenuta nel file *RMI_interfaceFile.java*) in cui vengono definiti i metodi invocabili in remoto dal client:

- Il metodo **elimina_occorrenze** accetta come parametro d'ingresso il nome del file, elimina tutte le occorrenze di caratteri numerici (da '0' a '9') all'interno di quello stesso file, e restituisce l'intero positivo del numero di eliminazioni effettuate (≥ 0) in caso di successo, oppure -1 in caso di insuccesso, ad esempio errori o altro. Il file deve essere cambiato sul server.
- Il metodo **lista_filetesto** accetta come parametro d'ingresso il nome del direttorio, e restituisce la lista dei nomi dei file di testo trovati, se ci sono. In caso di direttorio inesistente, prevedere una segnalazione di errore.

Si progettino le classi:

- **RMI_Server** (contenuta nel file *RMI_Server.java*), che implementa i metodi del server invocabili in remoto;
- **RMI_Client** (contenuta nel file *RMI_Client.java*), che realizza l'interazione con l'utente proponendo ciclicamente i servizi che utilizzano i due metodi remoti, e stampa a video i risultati, fino alla fine del file di input da tastiera.

Parte C

Progettare un **servitore multiservizio (usando la select)** che consenta di effettuare le operazioni remote per:

- **contare il numero delle linee che iniziano con una lettera maiuscola e contengono almeno uno specificato numero di occorrenze di un carattere indicato all'interno di tutti i file di testo presenti sul direttorio remoto,**
- **trasferire dal server al client tutti i file binari di un direttorio e dei suoi sottodirettori di primo livello, attraverso un unico processo figlio per sessione.**

utilizzando **un'unica connessione per ogni sessione cliente:** questo vincolo e' da intendersi come prioritario e fondamentale.

Più in dettaglio:

- Il **client_stream** è organizzato come un **processo filtro ciclico fino alla fine del file di input** e realizza la funzionalità di **trasferimento dal server al client di tutti i file binari di un direttorio e dei suoi sottodirettori di primo livello** utilizzando **socket stream** e **un'unica socket**. Per ogni richiesta, il client chiede all'utente il nome del direttorio, che invia al server, per poi ricevere i file salvandone il contenuto sul direttorio client.
- Il **client_datagram** è organizzato come un **processo filtro ciclico fino alla fine del file di input** e realizza la funzionalità **conteggio del numero di linee** all'interno di **tutti i file di testo** presenti sul direttorio (corrente) remoto. Per ogni richiesta, il client chiede all'utente e invia al server il carattere e il richiesto numero di occorrenze, quindi riceve l'esito e lo stampa a video.
- Il **server** multiservizio discrimina le richieste utilizzando la primitiva **select**: il **server gestisce in modo parallelo** la funzionalità di **trasferimento dal server al client di tutti i file binari di un direttorio e dei suoi sottodirettori di primo livello**, il figlio legge il nome del direttorio, quindi attua un protocollo per inviare tutti i file testo trovati nel direttorio e in tutti i suoi sottodirettori di primo livello al client. Per ogni richiesta di **conteggio del numero di linee** all'interno di **tutti i file di testo** presenti sul **direttorio remoto**, il server legge il carattere e il numero di occorrenze, quindi esegue il conteggio, scandendo ciascun file di testo riga per riga e contando il numero di righe che iniziano con una lettera maiuscola e contengono (in qualsiasi posizione della riga) esattamente lo specificato numero di occorrenze del carattere (cioè solo uguale a quelle indicate dall'utente), su ciascun file di testo trovato; quindi invia la risposta al client, rappresentata da un intero positivo che indica il numero totale di occorrenze contate (≥ 0) in caso di successo, -1 in caso di problemi.